Centro Universitário Nossa Senhora do Patrocínio

CEUNSP

Amanda de Souza Ferreira (RGM: 38962110)

Diogo Mello (RGM: 43078281)

Gabriel Silva de Almeida (RGM: 37338528)

Igor Ferguson Bertelli (RGM: 38019558)

Jaqueline Soares Mendes (RGM: 38834693)

Nathan Henrique do Prado (RGM: 37764357)

Detecção de Elementos Duplicados: Força Bruta ($O(n^2)$) vs. Parada Cedo

Salto, São Paulo

2026

Sumário

1. Introdução 3
2. Objetivo 3
3. Implementação do Código utilizando a linguagem C 4
   3.1. Arquivo `duplicatas.h` 4
   3.2. Arquivo `duplicatas.c` 5
   3.3. Arquivo `main.c` 6
4. Análise do Pior Caso 9
   4.1. Análise matemática 9
   4.2. Provando o Pior Caso 11
5. Análise do Melhor Caso 16
   5.1. Provando o Melhor Caso 16
6. Testes e Experimentos 17
   6.1. Resultados Teóricos Esperados 18
   6.2. Análise Comparativa 18
7. Conclusão 20

1. Introdução

A análise de complexidade de algoritmos é uma área fundamental da Ciência da Computação, pois permite estudar como o custo computacional de uma solução varia conforme o tamanho da entrada. Essa análise ajuda a compreender a eficiência dos algoritmos e a identificar situações em que determinada implementação pode exigir mais ou menos operações.

Este projeto aborda o problema da detecção de elementos duplicados em um vetor de números inteiros, utilizando um algoritmo baseado na comparação de pares de elementos. A implementação contabiliza o número de comparações realizadas durante a execução, permitindo investigar o comportamento do algoritmo em diferentes situações.

A análise considera dois cenários principais: o melhor caso, no qual a duplicata é encontrada logo na primeira comparação, e o pior caso, no qual todos os elementos são distintos e o algoritmo precisa examinar todos os pares possíveis. Além da implementação em linguagem C, o projeto utiliza testes experimentais e demonstrações matemáticas para relacionar o número de comparações observado às classes de complexidade assintótica.

2. Objetivo

O objetivo desse projeto é implementar e analisar experimentalmente um algoritmo de detecção de elementos duplicados em vetores, utilizando a contagem de comparações para investigar sua complexidade computacional no melhor e no pior caso. Temos alguns objetivos específicos para a resolução do projeto:

1. Implementar o algoritmo de detecção de duplicatas em linguagem C.
2. Contabilizar cada comparação entre elementos do vetor.

3. Verificar o comportamento do algoritmo quando uma duplicata é encontrada imediatamente.
4. Analisar o comportamento do algoritmo quando todos os elementos são distintos.
5. Executar testes com vetores de diferentes tamanhos.
6. Comparar os resultados experimentais com as funções de custo obtidas matematicamente.
7. Demonstrar formalmente as classes de complexidade assintótica por meio das notações Big O, Big Omega e Big Theta.
8. Relacionar os resultados obtidos com os conceitos teóricos de análise de algoritmos.

3. Implementação do Código utilizando a linguagem C

A implementação foi desenvolvida na linguagem C e organizada em três arquivos: **`main.c`**, **`duplicatas.c`** e **`duplicatas.h`**. Essa divisão separa a execução dos testes, a implementação do algoritmo e a declaração de sua interface, tornando o código mais organizado e facilitando sua manutenção.

O algoritmo utiliza duas estruturas de repetição para comparar pares de elementos do vetor. O primeiro laço seleciona um elemento de referência, enquanto o segundo percorre os elementos seguintes, evitando comparar um elemento consigo mesmo ou repetir pares que já foram analisados. A cada comparação entre dois valores, um contador é incrementado. Caso os valores sejam iguais, a função retorna imediatamente, interrompendo a busca; caso contrário, continua até verificar todos os pares possíveis.

3.1. Arquivo `duplicatas.h`

O arquivo `duplicatas.h` contém a declaração da função responsável pela detecção de elementos duplicados. Ele define a interface utilizada pelos

demais arquivos, informando os tipos de dados recebidos e o valor retornado

```c
#ifndef DUPLICATAS_H
#define DUPLICATAS_H

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

bool tem_duplicado(const int vetor[], size_t n, uint64_t *comparacoes);

#endif
```

pela função.

A função `tem_duplicado` recebe um vetor de números inteiros, seu tamanho e um ponteiro para o contador de comparações. Seu retorno é do tipo `bool`, indicando se uma duplicata foi encontrada. O uso de `size_t` permite representar o tamanho do vetor, enquanto `uint64_t` é utilizado para armazenar a quantidade de comparações realizadas.

As diretivas `#ifndef`, `#define` e `#endif` evitam que o cabeçalho seja incluído repetidamente durante a compilação.

3.2. Arquivo `duplicatas.c`

```c
#include "duplicatas.h"

bool tem_duplicado(const int vetor[], size_t n,
                   uint64_t *comparacoes)
{
    *comparacoes = 0;

    for (size_t i = 0; i + 1 < n; i++) {
        for (size_t j = i + 1; j < n; j++) {
            (*comparacoes)++;

            if (vetor[i] == vetor[j]) {
                return true;
            }
        }
    }

    return false;
}
```

O arquivo `duplicatas.c` contém a implementação do algoritmo. A função percorre o vetor por meio de dois laços de repetição, comparando cada elemento com os elementos que aparecem depois dele.

Inicialmente, o contador de comparações é zerado. O laço externo utiliza a variável `i` para selecionar o elemento de referência, enquanto o laço interno utiliza `j` para percorrer os elementos seguintes.

Antes de verificar se os valores são iguais, a instrução `(*comparacoes)++` incrementa o contador. Quando a condição `vetor[i] == vetor[j]` é verdadeira, a função retorna `true` imediatamente, encerrando a busca. Caso todos os pares sejam analisados sem encontrar valores iguais, a função retorna `false`. Essa implementação evita comparações redundantes entre pares de elementos e permite contabilizar o número de comparações efetivamente realizadas entre os valores do vetor.

3.3. Arquivo `main.c`

O arquivo `main.c` é responsável por executar os testes do programa. Nele são definidos os vetores utilizados para representar o melhor e o pior caso, além das quantidades esperadas de comparações. A função auxiliar `executar_caso` chama o algoritmo, compara os resultados obtidos com os valores esperados e apresenta um relatório no terminal.

```
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include "duplicatas.h"

static bool executar_caso(
    const char *nome,
    const int vetor[],
    size_t n,
```

```
    bool duplicata_esperada,
    uint64_t comparacoes_esperadas)
{
    uint64_t comparacoes = 0;

    bool encontrada =
        tem_duplicado(vetor, n, &comparacoes);

    bool correto =
        encontrada == duplicata_esperada &&
        comparacoes == comparacoes_esperadas;

    printf("\\n%s\\n", nome);
    printf("Vetor: [");

    for (size_t i = 0; i < n; i++) {
        if (i > 0) {
            printf(", ");
        }
        printf("%d", vetor[i]);
    }

    printf("]\\n");
    printf("Tamanho: %zu\\n", n);
    printf("Duplicata encontrada: %s\\n",
           encontrada ? "sim" : "nao");
    printf("Duplicata esperada: %s\\n",
           duplicata_esperada ? "sim" : "nao");
    printf("Comparacoes obtidas: %" PRIu64 "\\n",
           comparacoes);
    printf("Comparacoes esperadas: %" PRIu64 "\\n",
           comparacoes_esperadas);
    printf("Conferencia: %s\\n",
           correto ? "OK" : "FALHA");

    return correto;
}

int main(void)
{
    const int melhor[] =
        {0, 0, 2, 3, 4, 5, 6, 7, 8, 9};

    const int pior[] =
        {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};

    const size_t tamanho_melhor =
        sizeof melhor / sizeof melhor[0];

    const size_t tamanho_pior =
        sizeof pior / sizeof pior[0];

    const uint64_t comparacoes_pior =
        (uint64_t)tamanho_pior *
        (uint64_t)(tamanho_pior - 1) / 2;

    printf("Deteccao de elementos duplicados "
           "por comparacao de pares\\n");
```

```
    bool melhor_correto = executar_caso(
        "Melhor caso",
        melhor,
        tamanho_melhor,
        true,
        1
    );

    bool pior_correto = executar_caso(
        "Pior caso",
        pior,
        tamanho_pior,
        false,
        comparacoes_pior
    );

    if (melhor_correto && pior_correto) {
        printf("\\nOs dois casos produziram os "
               "resultados esperados.\\n");
        return EXIT_SUCCESS;
    }

    printf("\\nFoi encontrada uma divergencia "
           "na demonstracao.\\n");

    return EXIT_FAILURE;
}
```

A função `executar_caso` recebe os dados necessários para realizar um teste e verifica duas condições: se a detecção de duplicatas corresponde ao resultado esperado e se a quantidade de comparações coincide com o valor previsto. A função retorna `true` quando ambas as condições são satisfeitas e `false` caso contrário.

No `main`, o vetor melhor contém uma duplicata nas duas primeiras posições, permitindo testar a interrupção antecipada. Já o vetor pior possui apenas valores distintos, obrigando o algoritmo a examinar todos os pares. A expressão `sizeof vetor / sizeof vetor[0]`, aplicada aos vetores declarados no próprio `main`, determina a quantidade de elementos de cada vetor. Para o pior caso, o programa calcula a quantidade esperada de comparações por meio da fórmula `(n(n-1)/2)`.

Ao final, o programa retorna `EXIT_SUCCESS` quando os dois testes apresentam os resultados esperados ou `EXIT_FAILURE` quando ocorre alguma divergência.

4. Análise do Pior Caso

O pior caso ocorre quando o algoritmo percorre todas as comparações possíveis entre os elementos do vetor sem encontrar valores duplicados. Isso acontece quando todos os elementos são distintos, fazendo com que o algoritmo execute completamente os dois laços de repetição antes de retornar o resultado de que não existem duplicatas.

Na implementação desenvolvida, o laço externo seleciona cada elemento do vetor, enquanto o laço interno o compara com todos os elementos que aparecem posteriormente. Como nenhuma comparação identifica valores iguais, a instrução de retorno antecipado não é executada, e o algoritmo precisa verificar todos os pares possíveis.

4.1. Análise matemática

Considerando um vetor de tamanho n, o número de comparações realizadas diminui a cada iteração do laço externo. O primeiro elemento é comparado com os $n\text{-}1$ elementos seguintes; o segundo, com $n\text{-}2$; e assim sucessivamente, até que o penúltimo elemento seja comparado com o último. Sendo assim, vamos supor que $n = 6$:

$$5 + 4 + 3 + 2 + 1 = 15$$

$$\underline{1 + 2 + 3 + 4 + 5 = 15}$$

$$6 + 6 + 6 + 6 + 6 = 30$$

Vemos que o número de elementos é n = 6, enquanto o número de comparações se dá a partir de n–1, contendo um resultado de 15 comparações

e 30 ao somarmos as duas equações. De certa forma, podemos traduzir isso para uma fórmula matemática de soma:

$$S = \frac{(n-1) \cdot n}{2}$$

Substituindo:

$$S = \frac{(6-1) \cdot 6}{2}$$

$$S = \frac{5 \cdot 6}{2} \qquad S = \frac{30}{2}$$

$$S = 15 \text{ comparações}$$

Dessa forma, o número total de comparações pode ser representado pela seguinte soma:

$$C(n) = (n-1) + (n-2) + \ldots + 2 + 1$$

Utilizando a fórmula da soma dos n-1, podemos escrevê-la em nosso algoritmo dessa forma:

$$C(n) = \frac{n(n-1)}{2}$$

Sendo:

C = a fórmula de comparações

n = número de elementos

n – 1 = número de elementos a serem comparados com o primeiro elemento.

Simplificando a fórmula, podemos escrevê-la dessa forma:

$$C(n) = \frac{n^2 - n}{2}$$

A exemplo, utilizando o número 6:

$$C(6) = \frac{6^2 - 6}{2}$$

$$C(6) = \frac{36 - 6}{2}$$

$$C(6) = \frac{30}{2}$$

C (6) = 15 comparações

4.2. Provando o Pior Caso

Para a comparação no pior caso, precisamos distribuir a divisão para visualizarmos melhor a fórmula:

$$C(n) = \frac{n^2}{2} - \frac{n}{2}$$

Aqui com a quantidade exata de comparações no pior caso. Porém, a fórmula calcula o número de comparações; o Big O descreve um limite superior para o crescimento desse número. Para comparação:

10 elementos	45 comparações
20 elementos	190 comparações
100 elementos	4950 comparações

A exemplo:

$$C(4) = \frac{16}{2} - \frac{4}{2} = 8 - 2 = 6$$

Como essa fórmula prova que a complexidade é $O(n^2)$? O algoritmo em si faz 6 comparações, podemos dizer que 6 é menor ou igual a 16.

$$6 \leq 16? \; Sim!$$

A exemplo, 16 seria o valor de 4^2. Então, para $n = 4$, temos: $C(4) \leq 4^2$.

Mais um exemplo: $n = 5$

$$C(5) = \frac{25 - 5}{2}$$

$$C = 10$$

O limite superior de 5 seria: $5^2 = 25$.

$$Logo: 10 \leq 25$$

$$C(5) \leq 5^2$$

Provando que funciona para qualquer valor válido:

$$C(n) = \frac{n^2}{2} - \frac{n}{2}$$

O que queremos provar aqui é que o número de comparações do algoritmo é sempre menor ou igual ao quadrado do tamanho do valor. Tal que:

Tamanho (n)	Comparações C(n)	Limite (n²)
2	1	4
3	3	9
4	10	25
5	10	25
10	45	100

$C(n)$ fica abaixo do limite n^2. Algebricamente:

$$C(n) = \frac{n^2 - n}{2}$$

Parte de cima: $n^2 - n$. Como consideramos $n \geq 1$, sabemos que n é positivo. Subtrair um número positivo de n^2 faz o resultado diminuir. A exemplo, $n = 5$:

$$25 - 5 = 20$$

Sendo $20 \leq 25$. Portanto:

$$n^2 - n \leq n^2$$

Após isso, retomamos a divisão:

$$\frac{n^2 - n}{2} \leq \frac{n^2}{2}$$

Por que? Porque dividir os dois lados de uma desigualdade por um número positivo, mantém o sentido da desigualdade:

$N = 5$

$$\frac{20}{2} \leq \frac{25}{2}$$

$$10 \leq 12{,}5$$

Como $\frac{n^2}{2}$ é menor ou igual a n^2, conclui-se:

$$\frac{n^2 - n}{2} \leq \frac{n^2}{2}$$

Como a expressão ao lado esquerdo é justamente $C(n)$, temos:

$$C(n) \leq n^2$$

Essa é a expressão da demonstração algébrica. Nisso, utilizamos as propriedades da desigualdade para mostrar que o número de comparações não ultrapassa n^2.

Esse é o fim? Não. Temos um último passo: provar que essa afirmação pertence a $O(n^2)$! Precisamos encontrar constantes positivas c e n_0, tais que:

$$C(n) \leq c \cdot n^2$$

Para todo: $n \geq n_0$.

Sendo c = multiplicador que usamos no limite superior e n_0= tamanho a partir do qual nossa desigualdade precisa funcionar.

A partir disso, não precisamos adivinhar os n_0 , podemos encontra-los a partir do que foi provado, tal que: $c = 1$, já que: $c(n) \leq n^2$.

Mas a definição pede que o lado direito esteja escrito como $c \cdot n^2$. Podemos escrever: $C(n) \leq 1 \cdot n^2$.

Por fim, nada muda na matemática, pois multiplicar por 1 não altera o valor. Por isso, escolhemos $c = 1$. Poderíamos escolher $c = 2$? Sim, porque $2n^2$ é um limite ainda maior e a desigualdade também seria válida.

$$C(n) = \frac{n^2 - n}{2} \leq n^2$$

Agora, sabendo isso, n_0 precisa ser o tamanho a partir do qual nossa desigualdade precisa funcionar. A exigência é que desigualdade seja válida para todo $n \geq n_0$. Na prova, usamos o fato de que $n \geq 1$. Isso garante que n é positivo e conclui: $n^2 - n \leq n^2$.

Portanto, a demonstração funciona a partir de $n = 1$. Por isso: $n_0 = 1$.

Não quer dizer que o vetor precise ter apenas 1 elemento, mas que a desigualdade funciona para todos os tamanhos $n \geq 1$. Agora: c = 1 e $n_0 = 1$:

Substituindo: $C(n) \leq c \cdot n^2$

Obtemos: $C(n) \leq 1 \cdot n^2, n \geq 1$

Satisfazendo a definição de Big O. Logo:

$$C(n) \in O(n^2).$$

c	n	Funciona?
1	1	Sim
2	1	Sim
2	2	Sim
5	10	Sim
0	1	Não

Por que $c = 0$ daria erro? Porque $C(n) \leq 0 \cdot n^2 = 0$. Mas, a exemplo, para $n = 4$:

$$C(4) = \frac{4^2 - 4}{2} = 6$$

E $6 \leq 0$ é falso, a condição exige que seja positivo.

5. Análise do Melhor Caso

Utilizando um vetor mais simples: $\{7, 7, 3, 9, 2\}$, o código compara $\text{vetor}[0]$ com $\text{vetor}[1]$. Como ambas valem 7, encontra a duplicata e encerra imediatamente. O algoritmo realiza somente 1 comparação, mesmo que o vetor tenha milhares de elementos, desde que a duplicata esteja logo nas duas primeiras posições.

$$C_{melhor}(n) = 1$$

5.1. Provando o Melhor Caso

Para provar que uma função pertence a $\Omega\ (1)$, precisamos encontrar constantes positivas C e n_0, tais que:

$$C_{melhor}(n) = 1\ \geq c\ \cdot 1$$

Para todo $\text{n} \geq n_0$.

Observação: em prova de Big O, usamos $\leq$ para $\text{O}(\text{n}^2)$. Agora, em Big Omega, ou seja, $\text{O}(1)$, usamos $\geq$, pois queremos estabelecer um limite inferior. Sabendo que:

$$C_{melhor}(n) = 1$$

Podemos escolher: $\text{c} = 1$, $n_0 = 1$.

Substituindo na desigualdade:

$$1 \geq 1 \cdot 1$$

$$\text{Ou seja: } 1 \geq 1$$

Sendo assim, é verdadeiro. Como a desigualdade vale para todo $n \geq 1$, concluímos que:

$$C_{melhor}(n) = 1 \in \Omega(1)$$

Não necessita de somatório, pois o código encerra na primeira comparação.

Vamos a um exemplo onde dará falso: se $c = 2$ e $n_0 = 1$, a prova de $\Omega(1)$ funciona?

$$C_{melhor}(n) = 1 \geq c \cdot 1$$

$$1 \geq 2 \cdot 1$$

$$1 \geq 2? \text{ Falso!}$$

Em Big Omega, não basta escolher uma constante positiva. Ela precisa respeitar a desigualdade para todos os valores considerados.

$c = 1$, $n_0 = 1$, a desigualdade funciona: $1 \geq 1$.

Logo, prova concluída.

6. Testes e Experimentos

Os testes experimentais foram desenvolvidos com o objetivo de verificar o comportamento do algoritmo de detecção de elementos duplicados em diferentes situações de entrada. Para isso, foram considerados dois cenários principais: o melhor caso, no qual os dois primeiros elementos do vetor são iguais, e o pior caso, no qual todos os elementos são distintos.

A implementação utiliza um contador de comparações, incrementado sempre que dois elementos do vetor são comparados. Dessa forma, é possível avaliar o custo do algoritmo por meio de uma métrica objetiva, sem depender

exclusivamente do tempo de execução, que pode variar de acordo com o hardware e as condições de execução.

No melhor caso, o algoritmo identifica a duplicata na primeira comparação e encerra sua execução imediatamente. No pior caso, todas as comparações possíveis são realizadas antes de o algoritmo concluir que não existem elementos duplicados.

6.1. Resultados Teóricos Esperados

Considerando a função de custo demonstrada para cada cenário, podemos calcular previamente a quantidade esperada de comparações. No melhor caso, para vetores com pelo menos dois elementos, o número de comparações permanece constante:

$$C_{melhor}(n) = 1$$

No pior caso, quando todos os elementos são distintos, a quantidade de comparações é dada por:

$$C_{pior}(n) = \frac{n\,(n-1)}{2}$$

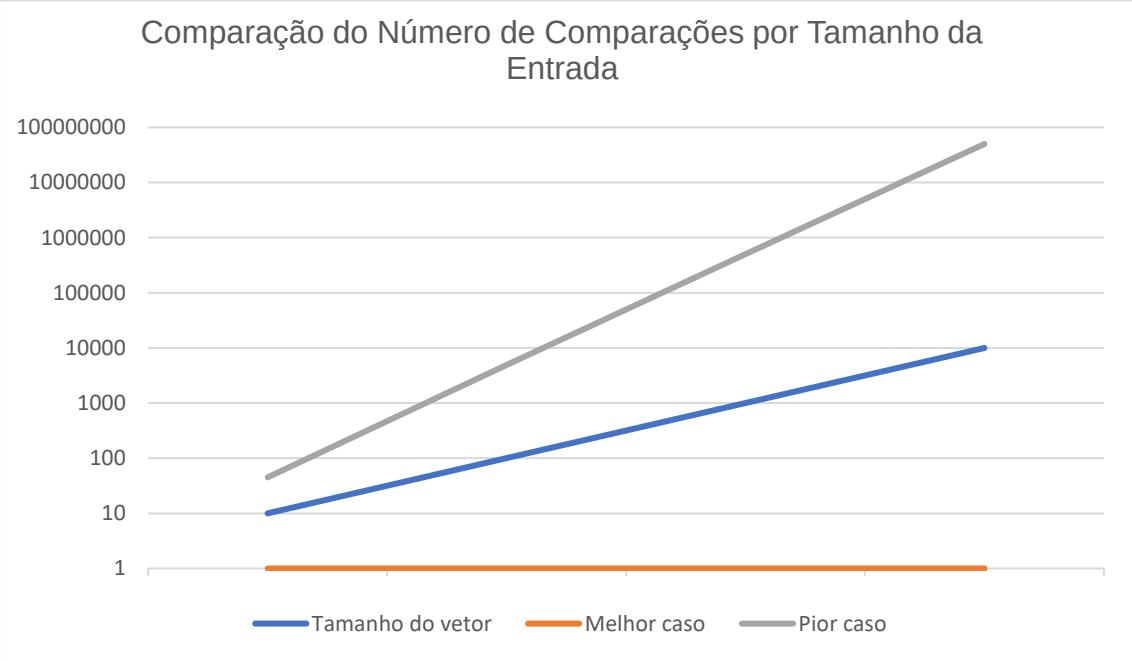
A tabela abaixo apresenta os valores teóricos esperados para diferentes tamanhos de entrada.

Tamanho do vetor (n)	Melhor caso	Pior caso
10	1	45
100	1	4.950
1.000	1	499.500
10.000	1	49.995.000

6.2. Análise Comparativa

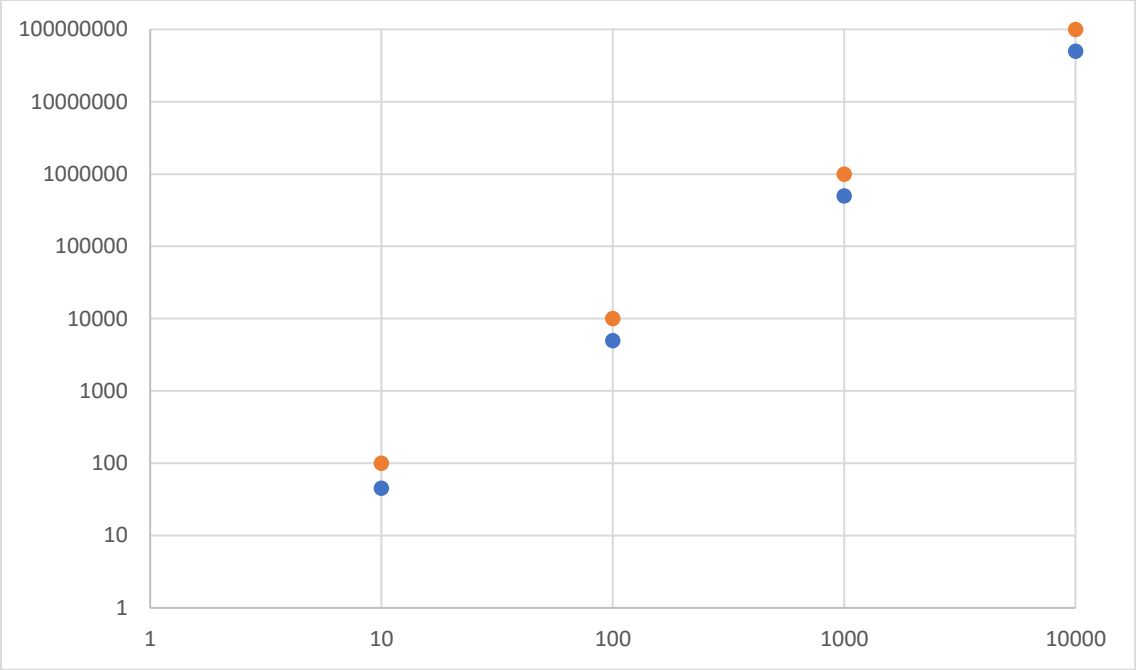
Os resultados teóricos evidenciam uma diferença significativa entre os dois cenários. Enquanto o melhor caso mantém apenas uma comparação, independentemente do tamanho do vetor, o pior caso apresenta um crescimento quadrático na quantidade de operações.

Essa diferença demonstra a importância da condição de parada antecipada implementada no algoritmo. Quando uma duplicata é encontrada logo no início, as comparações restantes são evitadas. Entretanto, quando todos os elementos são distintos, a ausência de uma duplicata impede a interrupção antecipada, tornando necessário examinar todos os pares possíveis. Assim, a análise experimental permite confrontar o comportamento observado durante a execução do programa com as funções de custo estabelecidas matematicamente, verificando a coerência entre a implementação e a análise teórica.



O gráfico evidencia a diferença entre o melhor e o pior caso do algoritmo. No melhor caso, o número de comparações permanece constante, igual a uma operação. No pior caso, o número de comparações aumenta quadraticamente conforme o tamanho do vetor cresce, em concordância com a função de custo

$C_{pior}(n) = \frac{n\,(n-1)}{2}$. Os valores apresentados são teóricos e devem ser confrontados com as contagens obtidas durante a execução do programa.



A comparação entre a função de custo $C_{pior}(n) = \frac{n\,(n-1)}{2}$ e a função de referência n^2 permite observar que ambas apresentam crescimento quadrático. Embora os valores de $C(n)$ sejam inferiores aos de n^2, a razão entre as funções tende a ½ à medida que o tamanho da entrada aumenta. Isso reforça a classificação do pior caso como $O(n^2)$.

7. Conclusão

O desenvolvimento deste projeto permitiu analisar o comportamento de um algoritmo de detecção de elementos duplicados em vetores, utilizando a linguagem C e a comparação direta entre pares de elementos. A implementação contou com um contador de comparações, possibilitando

quantificar o trabalho realizado pelo algoritmo e relacionar seu comportamento com os conceitos teóricos de análise de complexidade.

A análise do melhor caso demonstrou que, quando os dois primeiros elementos do vetor são iguais, o algoritmo identifica a duplicata na primeira comparação e encerra sua execução imediatamente. Dessa forma, o custo permanece constante, independentemente do tamanho do vetor, desde que existam pelo menos dois elementos. Esse comportamento foi classificado como $O(1)$.

Em contrapartida, a análise do pior caso considerou vetores cujos elementos são todos distintos. Nessa situação, o algoritmo precisa realizar todas as comparações possíveis entre pares de elementos, resultando na função de custo $C_{pior}(n) = \frac{n\,(n-1)}{2}$. Por meio da demonstração matemática, verificou-se que essa função apresenta complexidade $O(n^2)$, evidenciando o crescimento quadrático do número de comparações à medida que o tamanho da entrada aumenta.

Além disso, a utilização das notações assintóticas Big O, Big Omega e Big Theta contribuiu para compreender como os limites de crescimento de uma função de custo podem ser estabelecidos formalmente. A comparação entre $C(n)$ e n^2 também reforçou a classificação quadrática do pior caso, demonstrando a relação entre a função calculada e uma função de referência.

Por fim, o projeto evidenciou a importância de combinar a implementação prática com a análise matemática para compreender o desempenho de algoritmos. O uso de um contador de comparações permite avaliar o custo computacional de maneira objetiva, enquanto os testes possibilitam confrontar os resultados obtidos com os valores previstos teoricamente. Conclui-se que, embora a estratégia de comparação de pares seja simples e eficiente quando uma duplicata é encontrada logo no início, seu desempenho pode se tornar significativamente mais custoso em entradas sem elementos repetidos. Esse entendimento constitui uma base importante para

estudos futuros envolvendo estruturas de dados e algoritmos de detecção de duplicatas mais eficientes.